

Deep Learning Projects in Cybersecurity

ICCN- INE2

1. General Instructions

This assignment consists of one advanced project in cybersecurity using deep learning. The focus is on:

- Problem formulation and threat modeling
- Data understanding and preprocessing
- Experimental rigor
- Security reasoning and analysis
- Reproducibility

Coding is not the primary difficulty. All design choices must be justified.

2. Compute Constraints

- Maximum training time per experiment: 2 hours
- Use lightweight models only (MLP, LSTM, small CNN)
- Large-scale models are not allowed

3. Dataset Access

Students must use one of the provided datasets. If access is restricted, request access early or use a smaller subset.

4. Submission Format

- Report (PDF, max 10 pages)
- Code with README
- Reproducibility file with commands and dependencies

5. Evaluation Protocol

- Include a baseline model
- At least 3 experimental variations
- Use appropriate metrics (Precision, Recall, F1, PR-AUC, etc.)

6. Oral Defense

Each group will present and defend their work (10–15 minutes).

7. AI Usage Policy

AI tools are allowed, but all work must be understood and justified.

8. Reproducibility Requirements

- Fixed random seeds
- Document preprocessing
- Clearly define parameters

1 Project 1: Adversarially Robust Intrusion Detection

Project Overview

Core Idea: Build an intrusion detection system that remains effective even when attackers manipulate input features to evade detection.

Dataset:

- CIC-IDS2017: <https://www.unb.ca/cic/datasets/ids-2017.html>
- UNSW-NB15: <https://research.unsw.edu.au/projects/unsw-nb15-dataset>

Key Challenges:

- Defining realistic attacker capabilities
- Identifying feasible feature manipulations
- Evaluating robustness beyond accuracy

Tasks:

- Define threat model
- Analyze feature manipulability
- Train baseline model
- Design constrained adversarial attacks
- Implement defenses
- Evaluate robustness

Deliverables:

- Threat model document
- Attack and defense evaluation
- Robustness analysis report

Why it's hard: Requires combining deep learning with adversarial thinking and understanding real-world constraints on attackers.

Description

Modern IDS can be bypassed by small feature manipulations. This project explores robustness under adversarial conditions.

Steps

1. Define threat model
2. Identify manipulable features
3. Train baseline
4. Design attacks
5. Implement defenses
6. Analyze robustness

Expected Output

Robustness comparison and attack analysis.

2 Project 2: Malware Representation Learning

Project Overview

Core Idea: Learn latent representations of malware to uncover hidden structures such as families without labels.

Dataset:

- EMBER: <https://github.com/elastic/ember>
- Microsoft Malware: <https://www.kaggle.com/c/malware-classification>

Key Challenges:

- Designing meaningful representations
- Evaluating clustering without labels
- Interpreting learned embeddings

Tasks:

- Feature engineering
- Train embedding model
- Apply clustering
- Evaluate clustering
- Interpret clusters

Deliverables:

- Feature report
- Embedding visualization
- Cluster analysis

Why it's hard: Students must define similarity without supervision and justify representations.

Description

Focus on unsupervised learning for malware analysis.

Steps

1. Design representation
2. Train model
3. Cluster
4. Evaluate
5. Interpret

Expected Output

Embedding visualization and cluster interpretation.

3 Project 3: Insider Threat Detection

Project Overview

Core Idea: Detect anomalous user behavior over time using sequence modeling.

Dataset:

- CERT: <https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=508099>

Key Challenges:

- Modeling temporal behavior
- Handling imbalance
- Avoiding label leakage

Tasks:

- Build sequences
- Train models
- Compare methods
- Evaluate
- Analyze errors

Deliverables:

- Model comparison
- Error analysis

Why it's hard: Requires temporal reasoning and subtle anomaly detection.

Description

Detect insider threats using behavioral sequences.

Steps

1. Define anomaly
2. Build sequences
3. Train models
4. Evaluate
5. Analyze errors

Expected Output

Precision-recall and false positive analysis.

4 Project 4: Early Attack Detection

Project Overview

Core Idea: Detect cyber-attacks at early stages using weak signals.

Dataset:

- CSE-CIC-IDS2018: <https://www.unb.ca/cic/datasets/ids-2018.html>

Key Challenges:

- Defining early detection
- Handling imbalance
- Evaluating detection delay

Tasks:

- Define early detection
- Build truncated dataset
- Train model
- Evaluate delay
- Analyze trade-offs

Deliverables:

- Detection delay curves
- Trade-off analysis

Why it's hard: Requires formalizing vague concepts and designing new evaluation strategies.

Description

Focus on early detection before full attack manifestation.

Steps

1. Define early detection
2. Build dataset
3. Train model
4. Evaluate
5. Analyze trade-offs

Expected Output

Detection delay analysis.

5 Project 5: Explainable IDS

Project Overview

Core Idea: Make IDS decisions interpretable and assess explanation reliability.

Dataset:

- NSL-KDD: <https://www.unb.ca/cic/datasets/ns1.html>

Key Challenges:

- Interpreting black-box models
- Evaluating explanation stability
- Assessing adversarial risks

Tasks:

- Train model
- Apply explainability
- Evaluate stability
- Analyze security implications

Deliverables:

- Explanation analysis
- Security report

Why it's hard: Bridges interpretability and adversarial security.

Description

Explain and validate IDS decisions.

Steps

1. Train model
2. Explain predictions
3. Evaluate
4. Analyze risks

Expected Output

Explanation and security analysis.

6 Project 6: Data Poisoning Attacks

Project Overview

Core Idea: Study how training data can be manipulated to degrade model performance.

Dataset:

- UNSW-NB15: <https://research.unsw.edu.au/projects/unsw-nb15-dataset>

Key Challenges:

- Designing stealthy attacks
- Measuring impact
- Building defenses

Tasks:

- Define attack
- Train baseline
- Inject poison
- Evaluate impact
- Implement defense

Deliverables:

- Attack report
- Defense evaluation

Why it's hard: Requires attacker mindset and understanding of training dynamics.

Description

Analyze training-time attacks on ML systems.

Steps

1. Define attack
2. Train baseline
3. Poison data
4. Evaluate
5. Defend

Expected Output

Clean vs poisoned comparison.